

When Is Reuse Safe? Regime-Aware Policy Transfer for Recurring Drift in Network Telemetry

Neha Manoj
Ericsson Research
Bangalore, India
neha.manoj@ericsson.com

Abstract—Network telemetry drifts, and the standard fix, retraining, pays the full cost of rebuilding the model every time. But network regimes recur: a congestion profile, a radio configuration or an operator comes back. We ask whether a policy learned for a regime can be reused when that regime returns, and when doing so is safe. Regime-Aware Policy Transfer (RAPT) stores one ensemble policy per regime, retrieves it on recurrence, and refits it when rolling accuracy shows it no longer fits. On four network streams under a strict prequential protocol, RAPT uses 48–92% less adaptation CPU than full retraining. Macro-F1 is lower on all four streams, significantly so on two. The clearest failure is UGR’16, where regime identifiers recur but the feature-to-label mapping does not. Blind reuse drops macro-F1 to 0.836, and a parity refit that checks the reused policy’s recent accuracy recovers it to 0.928. So reuse keyed on regime identity alone is unsafe, and reuse with a performance check is not. On 5G Campus, a cost-aware refit configuration matches full retraining within measurement noise ($\Delta_{\text{macro-F1}} = +0.0022$, $p = 0.31$) at about 40% less adaptation CPU. Two other mechanisms failed under measurement and we report them: a relative-degradation trigger that cannot fire on a policy that was poor from the start, and an incremental learner that was worse on both accuracy and cost.

Index Terms—concept drift, network telemetry, online learning, adaptive ensembles, regime-aware learning, computationally efficient adaptation

I. INTRODUCTION

The data seen by network monitoring systems does not stay the same for long. Load, radio conditions, subscriber mobility and configuration changes each move the distribution, and a classifier fitted on a period of a 5G testbed or an ISP netflow archive will not stay accurate for very long. Retraining once performance drops is the standard answer. It works, but it has to be paid for again every time, and with high-rate telemetry the total spent on rebuilding models may end up above what it costs to serve predictions. This concern has already led to lightweight detection and adaptation frameworks for constrained devices [27]. Many good detectors exist for deciding *when* to adapt [4], [6], [7], [10], [12], [20], [22], but they make no use of how the change is structured. Conditions in networks often come back. A weekday congestion profile, a radio configuration or an operator-specific QoS regime may go away and return later. Some of the earliest work in the field was driven by recurring concepts [23], [25], and still the usual practice is to learn again from scratch, even though the model being thrown away may well be the right one.

The question we ask here is whether the discarded information can be kept and used. RAPT keeps a repository of adaptation policies, each indexed by an operating regime. At a regime boundary it looks up whether the incoming regime already has a stored policy. When it does, that ensemble is retrieved at close to zero adaptation cost, and when it does not, a new policy is trained and added to the repository. Since a retrieved policy can be wrong if the label relationship in that regime has changed, RAPT also follows its streaming accuracy and refreshes the policy if needed.

If regimes recur with stable label semantics, reuse keeps accuracy and removes most of the adaptation cost. If regimes recur but the feature-to-label relationship has moved, reuse brings back a stale policy and accuracy suffers, and the repair mechanism is the reason the approach still works in that setting. Our contributions are as follows:

- a regime-conditioned policy-transfer mechanism that stores and retrieves ensemble policies by regime;
- a monitoring and refresh strategy whose cost-aware configuration matches full-retraining accuracy at a fraction of the cost;
- an evaluation on four network streams under one prequential protocol, against four established drift detectors;
- a failure analysis covering harmful blind reuse, a trigger that cannot fire by construction, and an incremental learner that loses on both axes.

II. RELATED WORK

A. Concept and Data Drift

Drift in a data stream is usually sorted into two cases. Under covariate shift the marginal $P(X)$ changes while $P(Y | X)$ holds, and under concept drift it is the conditional $P(Y | X)$ that changes [12], [17]. Surveys have catalogued the taxonomies and adaptation strategies built up over the years [15]. The story starts with STAGGER and its recurring concepts [23], and with Widmer’s analysis of robustness against flexibility [25].

B. Drift Detection

ADWIN compares sub-window means inside an adaptive window [6], and Page-Hinkley is a sequential cumulative-sum procedure [20]. DDM and EDD follow the error rate together with its variance [4], [12]. RDDM was designed to give fewer false alarms [5], while the EDMA and EWMA

charts track an exponentially weighted moving average of the error [22]. A separate family needs no labels and tests the features directly. Examples use Hoeffding bounds [11], an incremental Kolmogorov–Smirnov statistic [10], or the earth-mover distance [7], and lightweight versions exist for constrained IoT deployments [27].

C. Adaptive and Online Learning

Online ensembles handle drift by updating their members as new data arrives. Dynamic weighted majority weights each learner by its recent accuracy and adds or drops learners along the way [16]; it grew out of earlier methods that minimize disagreement on drifting concepts [14]. Adaptive random forests pair online bagging with drift-aware tree replacement [13], and the accuracy-updated ensemble reweights its components when drift appears [8]. Similar designs have been proposed for social big-data and IoT streams [3], [26]. Single-pass incremental learners, of which Hoeffding trees are the standard example, keep memory use and per-example update cost bounded [9]. The detectors and incremental learners in our experiments come from the River library [19]. All of this work is about *how* a model gets updated. None of it considers fetching a model that is already good enough. In Section V-G we test one incremental learner against our approach, and on our stream it does not improve the accuracy–cost trade-off.

D. Transfer under Recurring Conditions

Transfer learning moves representations or parameters learned on a source task over to a related target task [21]. The idea of holding on to learned structure has a long history, with roots in work on the role of forgetting [18] and on the cost of reusing compiled knowledge chunks [24]. It saves a fitted policy with its regime identity and retrieves the two together. The decision it makes is whether any adaptation is needed, and how to adapt is left aside. With no recurrence, the mechanism collapses into ordinary retraining.

III. PROBLEM FORMULATION

We treat the data as a chronological stream $x_1, x_2, \dots$, where $x_t \in \mathbb{R}^d$ is a feature vector and $y_t \in \{1, \dots, C\}$ is its label. The label is unknown until the prediction for x_t has been issued. The stream is divided into fixed-size windows $W_1, \dots, W_T$, and each window W_k is tagged with a regime identifier $r_k \in \mathcal{R}$. A regime is a semantically defined operating condition, such as a network scenario or a time-of-day block. It is not a cluster label. A *policy* π is a fitted model plus its decision rule, and the repository is a mapping $\mathcal{P} : \mathcal{R} \rightarrow \pi$ that is filled in as new regimes show up. Whenever the stream moves from r_{k-1} to r_k , the controller picks an action $a_k \in \{\text{REUSE}, \text{TRAIN}\}$. It reuses $\mathcal{P}(r_k)$ when that exists, and trains a new policy on the recent buffer when it does not. Let $M(\pi, W)$ be a performance measure, which is macro-F1 in this paper, and let $c(\pi)$ be the cost of producing π in adaptation CPU time. The objective is to hold $M(\cdot, W_k)$ near the level a freshly trained policy would reach while making $\sum_k c(a_k)$ as small as possible.

Reuse rests on an assumption, namely that $P(Y \mid X, r_k)$ has not changed since the policy was stored. If the regime identifier returns after the conditional relationship has shifted, the stale policy gets applied and nothing flags it. The hard part is separating a recurring regime from a recurring label relationship.

IV. PROPOSED METHOD: RAPT

A. Architecture and Repository

RAPT sits between the stream and the model and makes one decision at each regime boundary. The active policy predicts each window, and the window is scored when its labels arrive. A change of policy happens only at a regime transition or when monitoring reports degradation.

We represent a regime with its semantic identifier and do not learn a clustering. What we give up is that the mechanism requires a usable regime signal; see Section VII. The repository holds one checkpoint per regime, made up of the fitted ensemble, its soft-voting weights, and timestamps for creation and last access. Checkpoints are immutable.

B. Matching and Transfer

Matching uses the regime identifier. On a transition to regime r , the controller tests whether $r \in \mathcal{P}$. The enhanced variant also applies a fingerprint-similarity gate, which requires the feature summary of the current window to be close to the fingerprint saved when the checkpoint was created. The effect is to relax the question “has this regime been seen?” into “has a sufficiently similar instance been seen?”

When reuse goes ahead, the retrieved ensemble becomes the active policy, and its soft-voting weights can be recalibrated using the most recent buffered window. Calibration computes the recent accuracy of the two components and renormalizes their weights. It covers at most the last 50 observations and is skipped when fewer than 20 are available, so its cost is small and fixed. If no checkpoint exists, we train a new policy from a class-anchored buffer and store it. The buffer holds a stratified slice of the training prefix. All adaptive models here use it in the same way, which stops a refit from collapsing onto a single class when the recent window has only one.

C. Monitoring, Refresh, and Fallback

We use two monitoring rules, and how they differ is one of our findings. The *absolute* rule triggers when rolling accuracy goes under a fixed floor. The *relative* rule triggers when accuracy has dropped by more than a fixed margin from a decayed reference, which is built from that policy’s own earlier accuracy. A relative rule cannot catch a policy that was never good, because the reference decays toward the poor level and so the gap it looks for never appears.

A refresh rebuilds the active policy from the recent buffer and replaces its checkpoint, and any later reuse then gets the refreshed policy. In the default refresh, the 100-tree ensemble is refit on a 1000-observation buffer. The cost-aware refresh has the same trigger but refits 20 trees on a 300-observation buffer. RAPT-Enhanced contains two additions. *Buffer-blended*

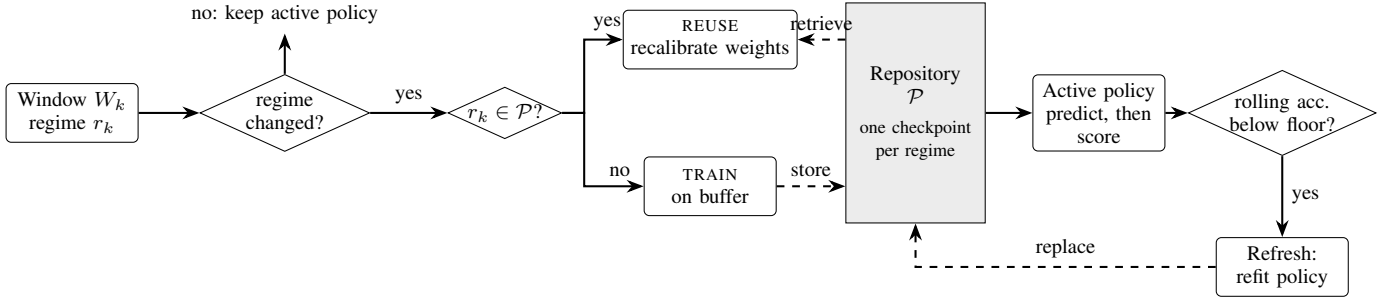


Fig. 1. RAPT overview. At each regime boundary the controller looks up the incoming regime in the repository and either reuses the stored policy or trains and stores a new one. The active policy predicts and is scored as labels arrive. A monitor refits the policy and replaces its checkpoint when rolling accuracy degrades. Solid arrows are control flow, dashed arrows are repository reads and writes.

novelty refitting refits a novel regime on a 1500-observation buffer, where the original used 500. *Selective parity refitting* refits a reused policy and replaces its checkpoint whenever rolling accuracy falls under a parity threshold over a short window. This targets stale reuse directly. When there is no checkpoint, or the similarity gate turns down the one that exists, the controller trains a fresh policy and stores it under the incoming regime identifier.

V. EXPERIMENTAL METHODOLOGY

A. Datasets

There are four datasets summarized in Tab.I, all of which contain data regarding Quality Of Service (QoS) in networks. All of these are based upon real-world network traces. The data in this paper were obtained from a number of sources. Data from a campus-based testbed is represented in “5G Campus Network QoS” and includes both delay metrics and classifications. This classification consists of three categories, GOOD, DEGRADED, and BAD. In order to determine whether the delay is classified as GOOD, DEGRADED, or BAD the 90th percentile value of the delay is determined in the next window. Each of the three scenarios will occur fourteen times. The second data set “UGR’16,” represents aggregated netflow data from an ISP. There are one hundred thirty-four features in the UGR’16 dataset. Due to the fact that there is very little labeled attack traffic in the calibration portion of the data (only blacklisted traffic), we chose to utilize the first thirty days of the labeled data rather than the calibration split. Since there is one labeled sample per minute, the total number of samples in the labeled data is forty-eight thousand. Time-of-day and weekday/weekend, when combined, results in twelve unique regimes and 168 unique recurrences. “NordicDat” is comprised of cross border LTE and 5G traces to be utilized for predictive quality of service [2]. Both of the above mentioned targets consist of three classes and are based upon the median delay of the next window. The regimes within “NordicDat” are based upon mobile operator. Finally, “5G NR End-to-End Latency” is a simulated dataset [1] of 499 windows with 500 packets each, distributed across four regimes (ABC-DACBDAB). Similar to “Campus,” the target for “5G NR” utilizes the same target metric as the campus stream.

TABLE I
DATASETS USED IN EVALUATIONS. RECURRENCES DENOTE OCCURRENCES WHERE A PARTICULAR REGIME IS REVISITED.

Property	Campus	UGR’16	Nordic	5G NR
Samples	1,799	43,200	91,455	249,500
Features	19	134	16	12
Classes	3	2	3	3
Windows	179	180	182	499
Window size	10	240	500	500
Regimes	3	12	3	4
Recurrences	14	168	17	6

B. Preprocessing, Ordering, and Leakage Control

All four streams undergo the exact same processing. First, all four streams undergo sorting by row according to times-tamp. Then, all identifier and constant columns are dropped. Finally, only those features that are numeric are retained. Any infinite values present within a stream are treated as missing. Any rows that lack a target value are deleted. Any remaining gaps are then filled using medians computed solely from the training prefix. Target thresholds and any scale-related statistics also derive from that prefix. None of the streams are shuffled. The prefix is the first twenty percent of windows: thirty-six for the three cross-dataset streams and ninety-nine for 5G NR. That prefix is completely excluded from evaluation, and that is the sole source of target thresholds. The order of operations is predict → evaluate → adapt for every window prior to its labels being used for adaptation. Regime annotation occurs directly from reading the stream. Therefore, no derivation of annotation can occur based upon the target.

C. Models And Detectors

Each of the five primary models utilizes exactly the same base architecture and the same retraining buffer. Therefore, any variation between them is due to their respective adaptation mechanisms only. The base is an ensemble of a random forest classifier and an extra-trees classifier, both having fifty trees of depth seven. Frozen is trained once on the prefix and receives no subsequent updates. Event-Driven will update when window accuracy falls below a fixed threshold. Full

Retraining will recreate/rebuild the model at every regime transition. RAPT and RAPT-Enhanced represent the controller(s) explained in Section IV.

ADWIN, Page-Hinkley, EDD and EDMA operate within the same retraining harness, with identical model, buffer, capacity and minimum-retrain-interval. They differ only by how they detect triggers. We pre-set their parameters in advance and did not adjust them via testing against our test streams (ADWIN $\delta = .002$; Page-Hinkley min = Thirty; THRESHOLD = Fifty; EDD WARNING = Two; DRIFT = Three; EDMA $\alpha = .2$ k = Two). Since none of our streams include true ground truth drift boundary indicators we report detected events, adaptation costs and predictive performance. Precision and Recall were not reported for our detectors.

D. Metrics And Statistics

Performance is evaluated through macro-F1, Accuracy, Precision and Recall measures. Costs may be measured in two fashions. Adaptation CPU denotes CPU time exclusively devoted to adaptation routines while Runtime indicates Total Streaming Time. A reduction in Adaptation Time does not necessarily translate into a proportional reduction in Total Runtime.

Each experiment had seeds [42,43,44,45,46]. Therefore we report Mean and Standard Deviation for each measure. For comparison purposes we use the Wilcoxon Signed-Ranked Test for paired comparisons along with Cohen's d . With respect to pairing by Window there would be approximately 143–146 observations. When paired by Seed we would have $n = 5$ with a minimum two-sided p -value available to us of .0625. If a result shows no significance we state that no difference was demonstrated between methods.

E. Predictive Performance

Table II holds the primary comparison, and Fig. 2 shows the macro-F1 values side by side.

On 5G Campus every model is close to saturation. Full Retraining has the best macro-F1 (0.9836 ± 0.0013), while RAPT and RAPT-Enhanced both land at 0.9381 ± 0.0016 . That is 0.0455 lower, and the gap is significant ($p = 2.6 \times 10^{-4}$, $d = -0.32$).

UGR'16 is where the assumptions of the mechanism fail, and it teaches us the most. Frozen has the highest macro-F1 (0.9690 ± 0.0029), which is consistent with attacks being sparse and clustered. RAPT reaches only 0.8360 ± 0.0262 . This is 0.1236 below Full Retraining ($p = 2.6 \times 10^{-11}$, $d = -0.67$) and the worst of the five models, despite its 133 reuses. The twelve regimes do recur, and their identifiers stay valid throughout the 30-day block, but the mapping from netflow features to attack labels does not stay put. When RAPT-Enhanced parity-refits a reused policy that has degraded, macro-F1 recovers to 0.9276 ± 0.0123 . The gain of 0.0917 ($p = 5.4 \times 10^{-9}$, $d = 0.56$) is the one statistically supported improvement of a variant over another on this stream.

NordicDat is hard to predict. Frozen scores 0.2779 ± 0.0224 , adaptation helps, and Full Retraining gets 0.4227 ± 0.0299

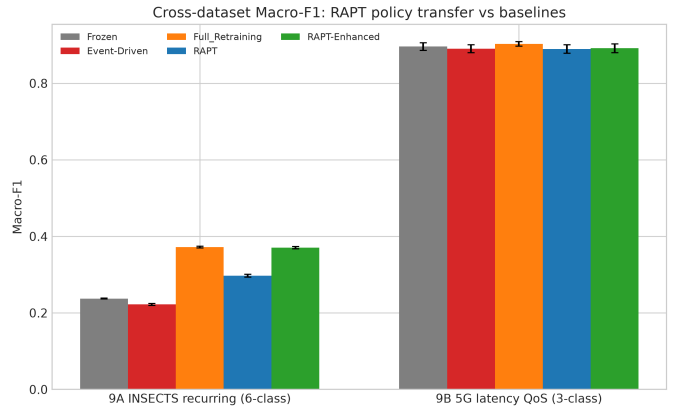


Fig. 2. Macro-F1 on the two largest cross-dataset streams. On UGR'16, RAPT tracks the frozen baseline and not the retrained models, which is the signature of stale policy reuse. On the 5G NR stream all five models are within 0.0133.

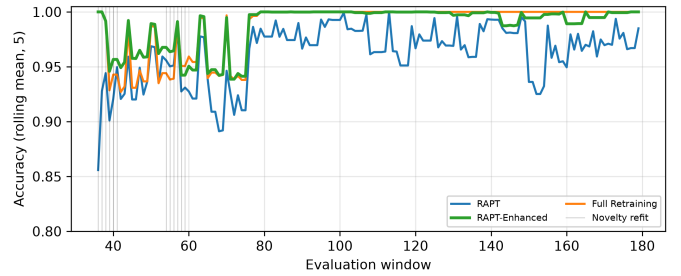


Fig. 3. Rolling accuracy over evaluation windows on UGR'16. RAPT sags once reused policies go stale, and the parity refit in RAPT-Enhanced pulls accuracy back toward Full Retraining. Vertical ticks mark parity refits.

against 0.3818 ± 0.0184 for RAPT. The deficit of 0.0409 is not significant ($p = 0.255$, $d = -0.12$). RAPT does have the highest accuracy on this stream (0.6172 ± 0.0127 against 0.5694 ± 0.0075). On 5G NR the five models are only 0.0133 apart. Taken together, the predictive results give no basis for saying that RAPT improves accuracy. Full Retraining has the best macro-F1 on three of the four streams, and RAPT sits below it on all four. The case for RAPT depends on cost, and on knowing when reuse is safe.

F. Computational Cost and Policy Reuse

Table II also lists adaptation and total cost, and Fig. 4 compares adaptation CPU. Relative to Full Retraining, the cost falls on every stream: from 1.4008 to 0.2292 s on 5G Campus (-83.6%), from 23.6318 to 1.9711 s on UGR'16 (-91.7%), from 2.2034 to 0.2594 s on NordicDat (-88.2%), and from 0.7636 to 0.3955 s on 5G NR (-48.2%). Full Retraining retrains at every regime transition, meaning 14, 144, 18 and 9 times. RAPT retrains 2, 11, 2 and 3 times and retrieves a stored policy 12, 133, 16 and 6 times.

G. Efficiency Ablation

Table III and Fig. 6 show the mechanism ladder on 5G Campus, in which each row adds one mechanism. RAPT-Full is the base controller with the similarity gate, and it scores

TABLE II

PRIMARY COMPARISON AND COST, MEAN OVER SEEDS 42–46. BOLD MARKS THE BEST MACRO-F1 PER STREAM; CPU AND RUNTIME IN SECONDS, EVENTS ARE COUNTS.

Dataset	Model	F1	Acc.	Prec.	Rec.	CPU (s)	Run (s)	Retr.	Reuse
5G Campus	Frozen	0.9364 $\pm$ 0.0023	0.9822 $\pm$ 0.0006	0.9379 $\pm$ 0.0025	0.9377 $\pm$ 0.0017	0.0000	1.2482	0	–
	Event-Driven	0.9636 $\pm$ 0.0006	0.9908 $\pm$ 0.0003	0.9637 $\pm$ 0.0008	0.9646 $\pm$ 0.0002	0.1080	1.3926	1	–
	Full Retraining	0.9836 $\pm$ 0.0013	0.9951 $\pm$ 0.0000	0.9838 $\pm$ 0.0015	0.9843 $\pm$ 0.0012	1.4008	2.6677	14	–
	RAPT	0.9381 $\pm$ 0.0016	0.9838 $\pm$ 0.0003	0.9393 $\pm$ 0.0017	0.9392 $\pm$ 0.0016	0.2292	1.4931	2	12
	RAPT-Enhanced	0.9381 $\pm$ 0.0016	0.9838 $\pm$ 0.0003	0.9393 $\pm$ 0.0017	0.9392 $\pm$ 0.0016	0.2337	2.1046	2	12
UGR'16	Frozen	0.9690 $\pm$ 0.0029	0.9899 $\pm$ 0.0007	0.9795 $\pm$ 0.0030	0.9662 $\pm$ 0.0028	0.0000	2.4914	0	–
	Event-Driven	0.8972 $\pm$ 0.0277	0.9898 $\pm$ 0.0013	0.9042 $\pm$ 0.0269	0.8955 $\pm$ 0.0280	0.5754	3.1303	3	–
	Full Retraining	0.9595 $\pm$ 0.0012	0.9873 $\pm$ 0.0007	0.9726 $\pm$ 0.0014	0.9580 $\pm$ 0.0015	23.6318	26.2627	144	–
	RAPT	0.8360 $\pm$ 0.0262	0.9635 $\pm$ 0.0182	0.8527 $\pm$ 0.0205	0.8336 $\pm$ 0.0294	1.9711	4.4990	11	133
	RAPT-Enhanced	0.9276 $\pm$ 0.0123	0.9880 $\pm$ 0.0017	0.9375 $\pm$ 0.0117	0.9251 $\pm$ 0.0123	2.7182	5.9329	11	133
NordicDat	Frozen	0.2779 $\pm$ 0.0224	0.4770 $\pm$ 0.0123	0.4215 $\pm$ 0.0226	0.2487 $\pm$ 0.0229	0.0000	2.5488	0	–
	Event-Driven	0.2547 $\pm$ 0.0177	0.4583 $\pm$ 0.0367	0.3909 $\pm$ 0.0208	0.2274 $\pm$ 0.0151	0.2533	2.8435	2.2	–
	Full Retraining	0.4227 $\pm$ 0.0299	0.5694 $\pm$ 0.0075	0.4938 $\pm$ 0.0233	0.4085 $\pm$ 0.0293	2.2034	4.7593	18	–
	RAPT	0.3818 $\pm$ 0.0184	0.6172 $\pm$ 0.0127	0.4703 $\pm$ 0.0185	0.3582 $\pm$ 0.0192	0.2594	2.8257	2	16
	RAPT-Enhanced	0.3783 $\pm$ 0.0321	0.5677 $\pm$ 0.0222	0.4600 $\pm$ 0.0307	0.3614 $\pm$ 0.0318	2.6492	5.8812	2	16
5G NR Lat.	Frozen	0.8961 $\pm$ 0.0099	0.9075 $\pm$ 0.0061	0.9314 $\pm$ 0.0083	0.8798 $\pm$ 0.0085	0.0000	1.7862	0	–
	Event-Driven	0.8903 $\pm$ 0.0107	0.9055 $\pm$ 0.0060	0.9475 $\pm$ 0.0085	0.8688 $\pm$ 0.0083	1.0406	2.7561	11	–
	Full Retraining	0.9027 $\pm$ 0.0059	0.9145 $\pm$ 0.0045	0.9478 $\pm$ 0.0053	0.8829 $\pm$ 0.0055	0.7636	2.4689	9	–
	RAPT	0.8894 $\pm$ 0.0110	0.9025 $\pm$ 0.0068	0.9258 $\pm$ 0.0103	0.8737 $\pm$ 0.0092	0.3955	2.1112	3	6
	RAPT-Enhanced	0.8915 $\pm$ 0.0117	0.9055 $\pm$ 0.0076	0.9348 $\pm$ 0.0106	0.8737 $\pm$ 0.0098	0.8609	2.5710	3	6

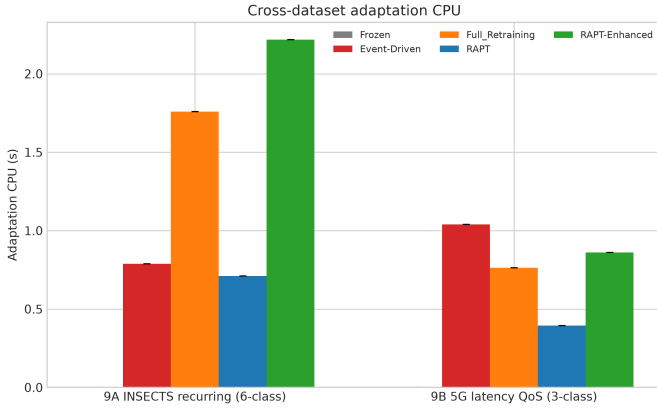


Fig. 4. Adaptation CPU on the two largest streams. The ordering is the reverse of Fig. 2: on UGR'16, RAPT uses an order of magnitude less adaptation time than Full Retraining and records lower macro-F1.

0.9587 $\pm$ 0.0000 macro-F1 at 0.4283 s. Replacing the absolute refit trigger with a relative-degradation trigger (RAPT-Evidence) leaves macro-F1 unchanged and fires *zero* refreshes. With an absolute floor (RAPT-Floor), 7.6 refreshes fire and the score is 0.9797 $\pm$ 0.0050 at 0.5865 s, higher than before but noisier. RAPT-Cheap keeps the periodic schedule and uses the cheaper refit. Its score is 0.9851 $\pm$ 0.0029 at 0.8477 s, compared with 0.9829 $\pm$ 0.0023 at 1.4063 s for Full Retraining. The macro-F1 difference is $\Delta = +0.0022$ with $p = 0.3125$, $d = 0.47$, and because the 95% interval $[-0.0036, +0.0080]$ includes zero we cannot claim a difference. The periodic refresh with a full refit scores 0.9847 $\pm$ 0.0028 at 2.7168 s. Set against it, the cheap refit keeps the accuracy and cuts adaptation CPU by roughly 69%. RAPT-Incremental is reported to have an average score of 0.9549 $\pm$ 0.0020 in approximately 0.5813 s. Although it has a lower accuracy compared to RAPT-

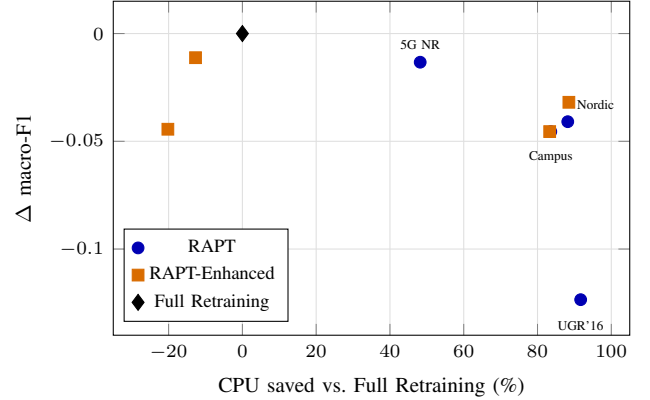


Fig. 5. Accuracy–cost trade-off on all four streams, measured against Full Retraining at the origin. Circles are RAPT and squares are RAPT-Enhanced, computed from Table II. Up and to the right is better. Squares left of zero cost more adaptation CPU than Full Retraining.

TABLE III

PERFORMANCE OF DIFFERENT CONFIGURATIONS OF RAPT USING THE 5G CAMPUS STREAM, AVERAGED OVER MULTIPLE RANDOM SEEDS.

Configuration	Macro-F1	Adapt. CPU	Refreshes
Full Retraining. (ref.)	0.9829 $\pm$ 0.0023	1.4063	–
RAPT-Full	0.9587 $\pm$ 0.0000	0.4283	0
RAPT-Evidence	0.9587 $\pm$ 0.0000	0.4260	0
RAPT-Floor	0.9797 $\pm$ 0.0050	0.5865	7.6
RAPT-Cheap	0.9851 $\pm$ 0.0029	0.8477	22
RAPT-Incremental	0.9549 $\pm$ 0.0020	0.5813	0

Full and a higher cost, the results clearly show a significantly lower cost as well. Therefore, the results indicate that the evidence supports matching levels of accuracy with greatly reduced costs. However, there is no indication that it provides better performance.

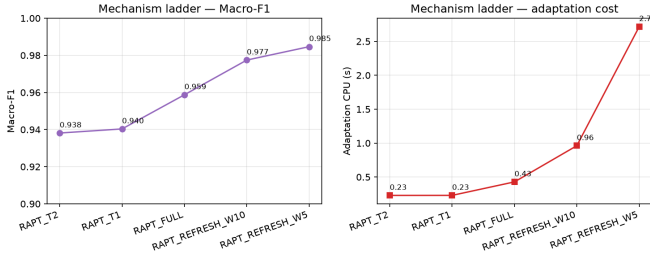


Fig. 6. Ablation ladder comparing the different configurations of RAPT, evaluated on the 5G Campus stream. As shown, RAPT-Cheap matches the full retrainer in terms of macro-F1 at around 40% less adaptation CPU, whereas RAPT-Incremental performs worse on both accuracy and cost.

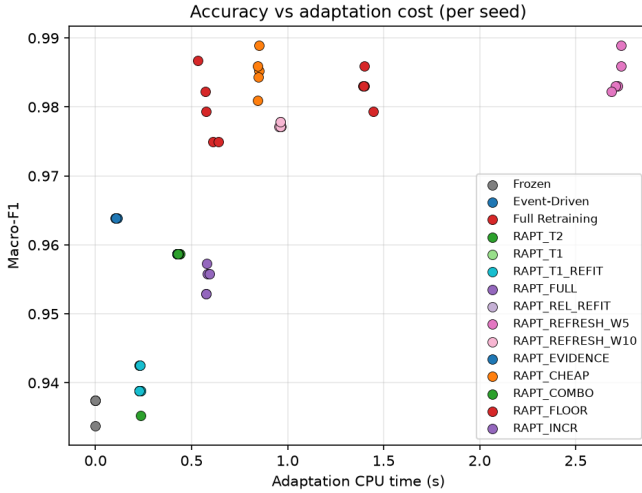


Fig. 7. Relationship between accuracy and cost of different versions of RAPT processed using data from the 5G Campus stream. We see that RAPT-Cheap achieves similar macro-F1 values at lower adaptation CPUs than those achieved by the full retrainer, while all detectors that were conservative did not generate any triggers and therefore coincided with the frozen baseline on both dimensions.

H. Determinants of Cost and Trigger Behavior

We determined the cheap refresh option through direct measurements. While holding the ensemble constant and varying each variable individually, we measured how long it took to refit when we changed factors such as tree count or buffer size. We saw that it takes around 22 ms to refit a model with 20 trees and around 104 ms to do the same with 100 trees. In addition, we observed that it takes around 80 ms to fit a model when we use a 200-observation buffer and around 104 ms when we use a 1000-observation buffer. Therefore, we can conclude that reducing the number of trees significantly reduces the time it takes to refit models. Specifically, we are able to reduce the refitting time by about four-fifths by reducing the number of trees used for modeling from 100 to 20.

In section V-G, we established that RAPT-Evidence generates no refreshes. An example of a relative trigger asks if the current accuracy value is at least some threshold below a degraded reference built upon the prior accuracy values generated from the prior policy. The reference will degrade

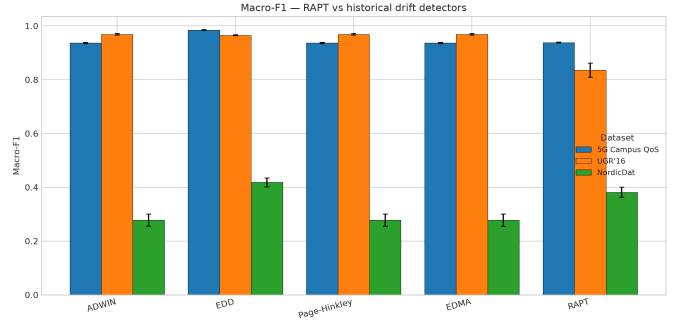


Fig. 8. Historical detectors comparison. Since ADWIN, Page-Hinkley and EDMA never fire they replicate the frozen baseline exactly. EDD fires most often and produces the highest macro-F1 at the highest adaptation cost. In contrast to these behaviors, RAPT has adapted less frequently than all but two of the methods studied here (and less accurately).

towards a poor level, and since this threshold is never met, the trigger will remain silent until the policy is no longer suitable. Compared to the threshold-based floor detector, which triggered 7.6 refreshes on data from the campus stream, RAPT-Floor generated no refreshes based on the relative rule.

I. Comparison to Historical Detectors

Of the four historical detectors included in our study, three produced no triggers during evaluation across all three test datasets. These detectors included ADWIN, Page-Hinkley and EDMA. Each of these detectors reproduced the frozen baseline by design. In contrast, EDD fired its trigger between 38 and 39 times per test dataset. This provided the best macro-F1 for either EDD or RAPT on two test datasets: EDD generated a macro-F1 of 0.9847 ± 0.0019 for the campus dataset and 0.4184 ± 0.0165 for the Nordic Dat dataset, respectively. Additionally, EDD incurred the greatest adaptation cost (3.98, 6.10 and 4.69 s) among all detectors included in our study. As depicted in Figure 8, these three operating points include:

RAPT operated differently than EDD and the other three detectors. It fired its trigger between 2 and 11 times for each test dataset, depending on the specific configuration being employed. In addition, RAPT incurred less adaptation cost than EDD in every case and less accurate results than EDD in none. Therefore, RAPT had the lowest adaptation cost among all detectors included in our study that could also adapt.

VI. DISCUSSION

The mechanism is worth deploying when there is recurrence. The cost saving appears wherever reuse occurs, but the accuracy penalty is concentrated on UGR'16, where regimes recur and their label relationship does not. There a reused policy is stale, and the parity refit earns its place by taking 0.8360 to 0.9276. For this reason reuse should come with a check on the reused policy's recent performance. Reusing on the regime identifier alone is unsafe when label semantics can drift independently of it.

VII. LIMITATIONS AND THREATS TO VALIDITY

Statistical power is the first concern. With five seeds, the smallest two-sided p -value attainable in a seed-level comparison is 0.0625, so some large effects cannot reach formal significance, and this applies most to the ablation results. Window-level tests assume that windows from one stream are independent, which inflates the effective sample size and weakens any claim meant to generalise. Other limitations come from the design itself. The mechanism relies on regime recurrence, which we measure and do not assume. A stream with no recurrence never exercises the repository, and a novel regime falls back to retraining with no advantage over the baseline. The RAPT-Floor threshold was calibrated on the evaluated stream, and the 20-tree cheap refit was chosen for this particular ensemble. Both would have to be revisited where the gap between pre- and post-drift data is larger, since a refit with reduced capacity could underfit. The repository is never pruned, and the approach takes regime similarity to be meaningful, which may fail if the regime signal is noisy.

VIII. CONCLUSION

Re-training is generally how to deal with “drift”, however, when things go back to normal, a portion of what you pay for is probably in your previous trained model. RAPT retains the policies learned for each regime and pulls them out again if/when that regime comes back. It will only train anew when there isn’t a suitable policy available and it will refresh a stored policy as soon as monitoring indicates that it doesn’t fit anymore. On four network streams, our experiments showed that adapting CPU usage was 48–92% lower compared to using full re-train. However, macro F-1 was lower on all four streams. Where regimes were going to occur again with different label relations (ie. same concepts, different labels), the blind reuse resulted in substantial damage; however, the parity-refit mechanism was able to fix this. We believe policy reuse is something to be deployed in cases where regimes recur with consistent or monitored label meanings. One thing we see as the next logical step in terms of adaptive reuse decisions, would be to test whether a retrieved regime has the same meaning as it did when the policy was stored.

REFERENCES

- [1] 5g nr end-to-end latency simulation dataset. Zenodo record 20035549. Accessed 2026.
- [2] Nordidat cross-border predictive qos dataset. Zenodo record 10964584, CC-BY-4.0. Accessed 2026.
- [3] Ahmad Abbasi, Abdul Rehman Javed, Chinmay Chakraborty, Jamel Nebhen, Wisha Zehra, and Zunera Jalil. Elstream: An ensemble learning approach for concept drift detection in dynamic social big data stream learning. *IEEE Access*, 9:66408–66419, 2021.
- [4] Manuel Baena-García, José del Campo-Ávila, Raul Fidalgo, Albert Bifet, Ricard Gavaldà, and Rafael Morales-Bueno. Early drift detection method. In *Proc. Int. Workshop on Knowledge Discovery from Data Streams*, pages 77–86, 2006.
- [5] Roberto S. M. Barros, Danilo R. L. Cabral, Paulo M. Gonçalves, and Silas G. T. C. Santos. Rddm: Reactive drift detection method. *Expert Systems with Applications*, 90:344–355, 2017.
- [6] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proc. SIAM Int. Conf. Data Mining (SDM)*, pages 443–448, 2007.
- [7] Tobias Brockhoff, Merih Seran Uysal, and Wil M. P. van der Aalst. Time-aware concept drift detection using the earth mover’s distance. In *Proc. Int. Conf. Process Mining (ICPM)*, pages 33–40, 2020.
- [8] Dariusz Brzeziński and Jerzy Stefanowski. Reacting to different types of concept drift: The accuracy updated ensemble algorithm. *IEEE Transactions on Neural Networks and Learning Systems*, 25(1):81–94, 2014.
- [9] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 71–80, 2000.
- [10] Denis Moreira dos Reis, Peter Flach, Stan Matwin, and Gustavo Batista. Fast unsupervised online drift detection using incremental kolmogorov-smirnov test. In *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, pages 1545–1554, 2016.
- [11] Isvani Frias-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jimenez, Rafael Morales-Bueno, Agustín Ortiz-Díaz, and Yailé Caballero-Mota. Online and non-parametric drift detection methods based on hoeffding’s bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823, 2015.
- [12] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. Learning with drift detection. In *Proc. Brazilian Symp. Artificial Intelligence (SBIA)*, pages 286–295, 2004.
- [13] Heitor M. Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfahringer, Geoff Holmes, and Talel Abdesslem. Adaptive random forests for evolving data stream classification. *Machine Learning*, 106(9–10):1469–1495, 2017.
- [14] David P. Helmbold and Philip M. Long. Tracking drifting concepts by minimizing disagreements. *Machine Learning*, 14(1):27–45, 1994.
- [15] Imen Khamassi, Moamar Sayed-Mouchaweh, Moez Hammami, and Khaled Ghédira. Discussion and review on evolving data streams and concept drift adapting. *Evolving Systems*, 9(1):1–23, 2018.
- [16] J. Zico Kolter and Marcus A. Maloof. Dynamic weighted majority: An ensemble method for drifting concepts. *Journal of Machine Learning Research*, 8(91):2755–2790, 2007.
- [17] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363, 2018.
- [18] Shaul Markovitch and Paul D. Scott. The role of forgetting in learning. In *Machine Learning Proceedings 1988*, pages 459–465. Morgan Kaufmann, 1988.
- [19] Jacob Montiel, Max Halford, Saulo Martiello Mastelini, Geoffrey Bolmier, Raphael Sourty, Robin Vaysse, Adil Zouitine, Heitor Murilo Gomes, Jesse Read, Talel Abdesslem, and Albert Bifet. River: machine learning for streaming data in python. *Journal of Machine Learning Research*, 22(110):1–8, 2021.
- [20] Ewan S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.
- [21] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359, 2010.
- [22] Gordon J. Ross, Niall M. Adams, Dimitris K. Tasoulis, and David J. Hand. Exponentially weighted moving average charts for detecting concept drift. *Pattern Recognition Letters*, 33(2):191–198, 2012.
- [23] Jeffrey C. Schlimmer and Richard H. Granger. Incremental learning from noisy data. *Machine Learning*, 1(3):317–354, 1986.
- [24] Milind Tambe and Allen Newell. Some chunks are expensive. In *Machine Learning Proceedings 1988*, pages 451–458. Morgan Kaufmann, 1988.
- [25] Gerhard Widmer. Combining robustness and flexibility in learning drifting concepts. In *Proc. European Conf. Artificial Intelligence (ECAI)*, pages 468–472, 1994.
- [26] Li Yang, Dimitrios Michael Manias, and Abdallah Shami. Pwpae: An ensemble framework for concept drift adaptation in iot data streams. In *Proc. IEEE Global Communications Conf. (GLOBECOM)*, pages 1–6, 2021.
- [27] Li Yang and Abdallah Shami. A lightweight concept drift detection and adaptation framework for iot data streams. *IEEE Internet of Things Magazine*, 4(2):96–101, 2021.